

Objetivo: Comprender el concepto de Tipo de Dato Abstracto (TDA), su utilización e implementación.

Introducción

Los Tipos de Dato Abstracto (TDA) representan una de las nociones fundamentales en el campo de la ciencia de la computación, particularmente en el desarrollo de software.

La importancia de los TDAs en la programación radica en su capacidad para promover la abstracción de datos, uno de los pilares de la programación moderna. La abstracción de datos les permite a los desarrolladores concentrarse en qué hacen los datos (sus operaciones) en lugar de cómo se almacenan o representan internamente. Esta separación entre la interfaz (las operaciones que se pueden realizar con los datos) y la implementación (cómo se realizan estas operaciones) es crucial para el desarrollo de software, ya que promueve la modularidad y la reutilización del código. La Figura 1 ilustra esta idea.

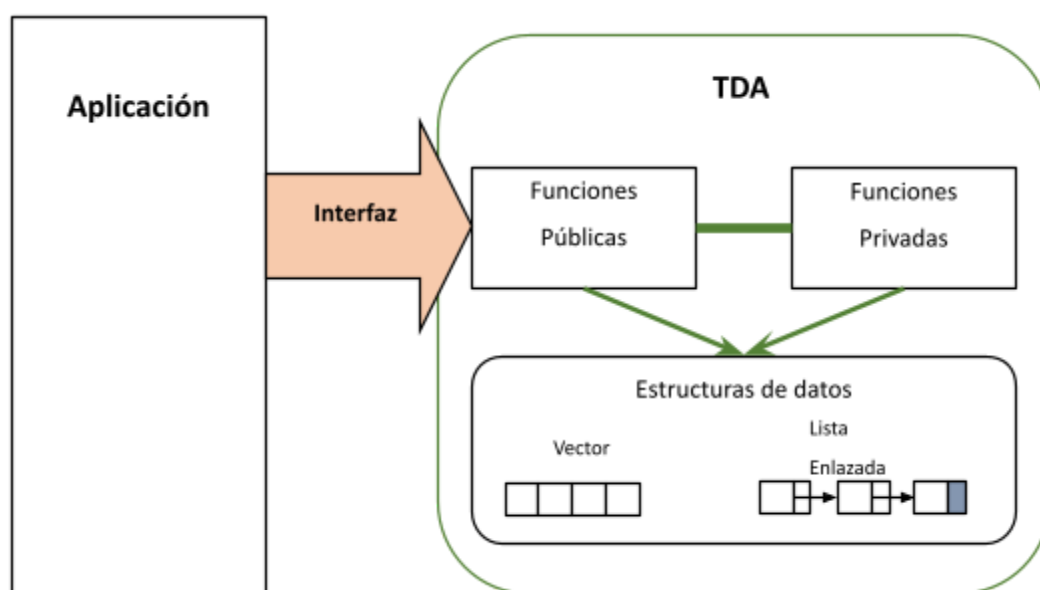




Figura 1. Abstracción de la implementación provista por un TDA.

Al utilizar TDAs, los programadores pueden cambiar la implementación interna de un TDA sin afectar el código que utiliza ese TDA, lo que facilita la mantenibilidad y la escalabilidad de los sistemas de software.

Definición

Un Tipo de Dato Abstracto (TDA) es una abstracción que especifica un conjunto de valores y un conjunto de operaciones que pueden realizarse sobre esos valores. La distinción clave de un TDA es que oculta los detalles de implementación de los datos y solo revela las operaciones que pueden realizarse sobre ellos, junto con sus comportamientos esperados.

Características Principales

1. **Abstracción:** El TDA proporciona una abstracción al usuario al definir un conjunto de datos y operaciones sin exponer los detalles internos de cómo se implementan.
2. **Encapsulamiento:** Los datos y las operaciones que se realizan sobre ellos se encuentran contenidos dentro de una interfaz bien definida. Esto ayuda a prevenir el acceso no autorizado a los datos y asegura la coherencia de los mismos.
3. **Independencia de Implementación:** Los usuarios de un TDA no necesitan conocer los detalles internos de su implementación. Esto permite que la implementación subyacente pueda cambiar sin afectar a los usuarios que solo interactúan a través de la interfaz definida.



4. **Reutilización:** Al ocultar los detalles de implementación, los TDAs promueven la reutilización del código. Diferentes implementaciones pueden proporcionar las mismas operaciones sobre los datos subyacentes, lo que facilita la adaptación del TDA a diferentes necesidades.

Ejemplos Comunes

- **Pilas (Stacks):** Un TDA que permite el acceso solo al elemento más recientemente añadido (LIFO - Last In, First Out).
- **Colas (Queues):** Un TDA que permite el acceso solo al elemento más antiguo añadido (FIFO - First In, First Out).
- **Listas Enlazadas (Linked Lists):** Un TDA que representa una secuencia de elementos donde cada elemento está vinculado al siguiente.
- **Árboles (Trees):** Un TDA que representa una estructura jerárquica de datos donde cada elemento tiene uno o más elementos secundarios.

Implementación

La implementación de un TDA implica definir una interfaz pública que incluya las operaciones permitidas y luego proporcionar una o más implementaciones de esas operaciones. Las implementaciones pueden variar en eficiencia y complejidad, pero siempre deben cumplir con el contrato establecido por la interfaz pública.

Tipo de Dato Abstracto (TDA) vs Estructuras de Datos (ED)

Veamos las diferencias entre un TDA y una ED.



Según definimos anteriormente, un **tipo de dato abstracto** es un modelo acompañado de una serie de operaciones que se pueden realizar sobre los datos que se ajustan a ese modelo.

Por otro lado, una **estructura de datos** es una forma específica de organizar y almacenar datos en la memoria de una computadora, junto con un conjunto de operaciones que acceden y transforman (cuando es necesario) dichos datos. Las estructuras de datos proporcionan un medio para representar datos de manera eficiente y permiten realizar operaciones como búsqueda, inserción, eliminación y actualización de datos de manera efectiva.

La diferencia está en el enfoque. Las estructuras de datos se enfocan en la organización física de los datos dentro de la memoria de la computadora, en cambio, los TDAs se enfocan en el comportamiento lógico de los datos, abstrayéndose de los detalles de implementación.

Como ejemplo, tomemos el caso de un TDA Cola. Desde la *perspectiva del TDA*, sabemos el primer dato que ingresa va a ser el último que egresa, solo se va a poder ver el frente de la cola, van a existir 7 primitivas para dicho TDA (crear_cola, cola_llena, encolar, desencolar, ver_frente, cola_vacia, vaciar_cola), etc. En definitiva, el comportamiento esperado de la cola. Ahora bien, desde la *perspectiva de la ED*, se deberá tener en cuenta, por ejemplo, si es una implementación estática o dinámica, ya que de ello depende la gestión de memoria y la implementación de las primitivas.

Ejemplo de aplicación

Vamos a presentar la siguiente situación: *tenemos un conjunto de números enteros y debemos almacenarlos en algún tipo de estructura, de manera tal que a medida que se vayan almacenando lo hagan de manera ordenada. Además, debe ser posible mostrar*



los elementos almacenados, accederlos para extraerlos y se pueda saber la cantidad de elementos almacenados. En la Figura 2 se muestra cómo evolucionará la estructura de datos planteada.

A primera vista se puede ver que se trata de un arreglo unidimensional, con la variación de la inserción en orden y la posibilidad de llevar la cuenta de la cantidad de elementos insertados en el arreglo.

Les propongo tomarse unos minutos para analizar el requerimiento y diagramar una solución teniendo en cuenta la Figura 1.

Para resolver el requerimiento planteado, diseñaremos una solución basada en un TDA al que denominaremos Vector. Ahora, dividamos el enfoque desde los puntos de vista de un TDA y de la ED.

Números a ingresar: 5, 3, 9, 1, 5

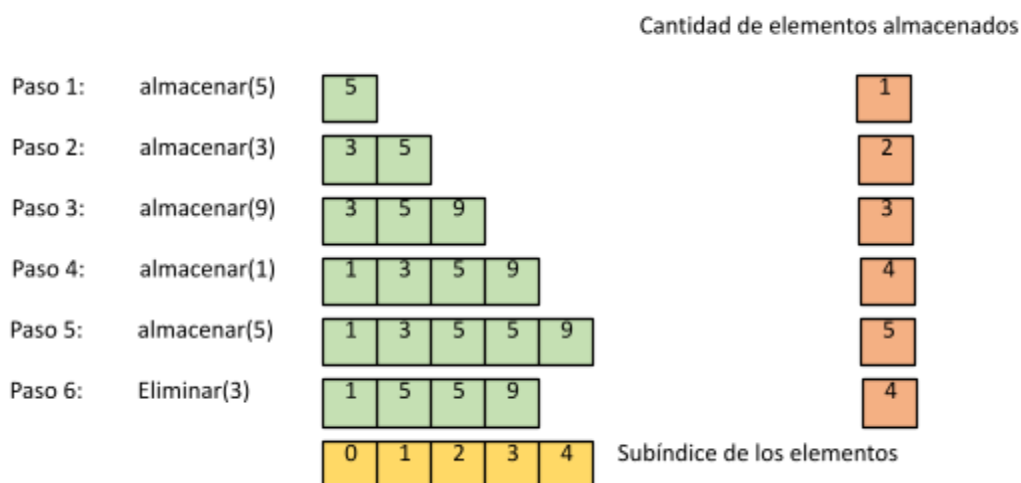


Figura 2. Abstracción de de la estructura de almacenamiento cuando se invocan las primitivas de almacenar y eliminar.



Enfoque orientado al Tipo de Datos Abstractos:

Nos piden que carguemos el listado de números. Para ello utilizaremos el TDA vector. Visto desde este enfoque, el problema se resuelve invocando las primitivas del TDA en el orden correcto (crear el vector, insertar los elementos, etc). No nos interesa cómo se encuentra implementado el TDA, solo nos enfocamos en **cómo se comporta**. La implementación del TDA pudo haber sido realizada por otro desarrollador o equipo de desarrollo. Ese es el fuerte de los TDA's en general, nos permiten enfocarnos en el problema puntual (insertar los datos en una estructura de manera ordenada). El TDA es una herramienta más para la resolución del requerimiento.

Nuestro TDA vector tendrá disponibles las siguientes operaciones:

- Crear vector
- Vector vacío
- Vector lleno
- Insertar en orden
- Destruir vector
- Eliminar elemento
- Mostrar vector
- Ver cantidad de elementos insertados

Enfoque orientado a la Estructura de Datos:

Situados en este enfoque, debemos pensar en cómo se implementará el TDA, teniendo en mente, por ejemplo, si se utilizará memoria estática o dinámica, si se implementará sobre un arreglo o sobre una lista enlazada. Aquí se tiene en cuenta **cómo funciona** el TDA puertas adentro.

Implementación del TDA Vector de ejemplo

Implementaremos una versión del TDA una sobre memoria estática, Para ello, los elementos insertados en el TDA serán almacenados en una arreglo de memoria unidimensional (un vector). En la Figura 3 se puede ver cómo se encuentra organizado el proyecto. El archivo *vector.h* contiene las definiciones del TDA y en el archivo *vector.c* se encuentran las implementaciones de las primitivas del TDA.

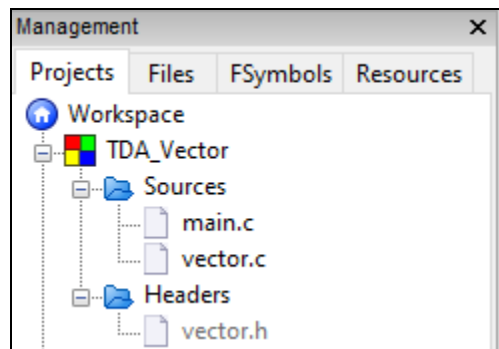


Figura 3. Proyecto en el cual se ha implementado el TDA Vector.

En la Figura 4 se puede ver el código del archivo *main.c*.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "vector.h"
4
5  int main()
6  {
7      tVector v;
8      int elemento;
9      int subindiceElemEliminar;
10
11     printf("Creando vector...\n");
12     crearVector(&v);
13
14     if(!vectorLleno(&v))
15         insertarEnOrden(&v, 5);
16
17     mostrarVector(&v);
18     insertarEnOrden(&v, 3);
19     mostrarVector(&v);
20     insertarEnOrden(&v, 9);
21     mostrarVector(&v);
22     insertarEnOrden(&v, 1);
23     mostrarVector(&v);
24     insertarEnOrden(&v, 5);
25     mostrarVector(&v);
26
27     subindiceElemEliminar = 1;
28     printf("Eliminando el elemento con subindice %d\n", subindiceElemEliminar);
29     eliminarElemento(&v, subindiceElemEliminar, &elemento);
30     printf("Elemento extraido: %d\n", elemento);
31     printf("Vector luego de extraer elemento:\n");
32     mostrarVector(&v);
33
34     printf("Cantidad de elementos insertados: %d\n", verCantidadElementos(&v));
35     printf("Destruyendo el vector...\n");
36     destruirVector(&v);
37
38     if (vectorVacio(&v))
39         printf("El vector se encuentra vacio.");
40     else
41         printf("El vector no se encuentra vacio.");
42
43     return 0;
44 }
45
```




Figura 4. Implementación de la solución de almacenamiento de los números.

Allí se pueden ver las invocaciones a las diferentes primitivas del TDA que resuelven el requerimiento. Es importante destacar que se está accediendo al TDA Vector a través de sus primitivas definidas en la interfaz, permitiendo abstraerse de su implementación. como se encuentra implementado el TDA. Podemos destacar las siguientes líneas de código en la función *main* (en el Apéndice 1, apartado Archivo vector.c se pueden ver las implementaciones de las primitivas):

Línea 7: se declara la variable **v** del tipo **tVector**. Las primitivas del TDA se invocarán pasando esta variable como argumento.

Línea 12: se inicializa la variable **v** con la primitiva **crearVector**. Esta primitiva se encarga de inicializar las variables internas del TDA. **Pregunta:** ¿por qué se pasa como argumento de la primitiva la variable **v** precedida por el operador **&** (**&v**) ?

Línea 14: se invoca a la primitiva **vectorLleno** para verificar si el vector se encuentra lleno antes de realizar una inserción en él.

Línea 15: se inserta un valor en el vector a través de la primitiva **insertarEnOrden**. El primer argumento es el vector, y el segundo el elemento a insertar. En este caso el elemento a insertar es un entero. ¿Qué se debería tener en cuenta si el elemento a insertar fuera, por ejemplo, un elemento del tipo de dato "tPersona"? ¿Qué sucede cada vez que se inserta un nuevo elemento ? Ver la implementación de la primitiva.

Líneas 17 a 25: se muestra el contenido del vector y se insertan nuevos elementos en él.

Línea 29: se muestra como invocar a la primitiva para eliminar un elemento. Nótese que a través del tercer argumento de la primitiva se estaría devolviendo el elemento eliminado (este es un ejemplo del uso de punteros). Si bien en esa línea no se está



teniendo en cuenta, la primitiva retorna 1 o 0 según se haya eliminado o no el elemento con subíndice 1 dentro del vector.

Línea 36: aquí se invoca al primitiva que destruye el vector. Es importante destacar que en este caso en particular, el TDA se encuentra implementado sobre memoria estática. Para eliminar el vector alcanza con asignar 0 a la variable `elementosInsertados`, sin embargo, si se encontrara implementado sobre memoria dinámica, habría que liberar los recursos tomados para alojar los elementos insertados en el TDA.

Línea 38: aquí se ejemplifica el uso de la primitiva ***vectorVacío***.

En la Figura 5 se ve la salida por pantalla de las operaciones realizadas sobre el TDA Vector.

```
Creando vector...
5
3 5
3 5 9
1 3 5 9
1 3 5 5 9
Eliminando el elemento con subíndice 1
Elemento extraído: 3
Vector luego de extraer elemento:
1 5 5 9
Destruyendo el vector...
El vector se encuentra vacío.

Process returned 0 (0x0)   execution time : 1.157 s
Press any key to continue.
```

Figura 5. Salida por pantalla con la evolución de los datos en el TDA Vector.

En el Apéndice 1 se puede ver la implementación completa del TDA Vector que se ha presentado como ejemplo.



Ejercicios propuesto

Modificar el ejemplo anterior para que sea implementado en memoria dinámica.

Apéndice 1

Archivo **vector.h**

```
vector.h X
1  #ifndef VECTOR_H_INCLUDED
2  #define VECTOR_H_INCLUDED
3
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  #define MAX_SIZE    5
8  #define VERDADERO   1
9  #define FALSO       0
10 #define ERROR       -1
11
12 typedef struct
13 {
14     int datos[MAX_SIZE];
15     int elementosInsertados;
16 } tVector;
17
18 void crearVector(tVector *v);
19 int vectorVacio(const tVector *v);
20 int vectorLleno(const tVector *v);
21 int verCantidadElementos(const tVector *v);
22 int insertarEnOrden(tVector *v, int elemento);
23 void destruirVector(tVector *v);
24 int eliminarElemento(tVector *v, int n, int *elemento);
25 void mostrarVector(const tVector *v);
26
27 #endif // VECTOR_H_INCLUDED
```



Archivo **vector.c**

```
vector.c X
1      #include "vector.h"
2
3      void crearVector(tVector *v)
4      {
5          v->elementosInsertados = 0;
6      }
7
8      int vectorLleno(const tVector *v)
9      {
10         return v->elementosInsertados == MAX_SIZE;
11     }
```



```
12
13  int verCantidadElementos(const tVector *v)
14  {
15      return v->elementosInsertados;
16  }
17
18  int vectorVacio(const tVector *v)
19  {
20      return v->elementosInsertados == 0;
21  }
22
23  int insertarEnOrden(tVector *v, int elemento)
24  {
25      int i;
26      int j;
27
28      if(v->elementosInsertados == MAX_SIZE)
29      {
30          return FALSO;
31      }
32
33      i = 0;
34      while(i < v->elementosInsertados && v->datos[i] < elemento)
35      {
36          i++;
37      }
38
39      for(j = v->elementosInsertados; j > i; j--)
40      {
41          v->datos[j] = v->datos[j - 1];
42      }
43
44      v->datos[i] = elemento;
45      v->elementosInsertados++;
46
47      return VERDADERO;
48  }
49
50  void destruirVector(tVector *v)
51  {
52      v->elementosInsertados = 0;
53  }
```



```
54
55  int eliminarElemento(tVector *v, int n, int *elemento)
56  {
57      if (n < 0 || n >= v->elementosInsertados)
58      {
59          return ERROR; // Indica un error, n no es válido
60      }
61
62      *elemento = v->datos[n];
63
64      for (int i = n; i < v->elementosInsertados - 1; i++)
65      {
66          v->datos[i] = v->datos[i + 1];
67      }
68
69      v->elementosInsertados--;
70
71      return VERDADERO;
72  }
73
74  void mostrarVector(const tVector *v)
75  {
76      int i;
77      for(i = 0; i < v->elementosInsertados; i++)
78      {
79          printf("%d ", v->datos[i]);
80      }
81      printf("\n");
82  }
83
```



Índice de Figuras

<i>Figura 1. Abstracción de la implementación provista por un TDA.</i>	1
<i>Figura 2. Abstracción de de la estructura de almacenamiento cuando se invocan las primitivas de almacenar y eliminar.</i>	5
<i>Figura 3. Proyecto en el cual se ha implementado el TDA Vector.</i>	6
<i>Figura 4. Implementación de la solución de almacenamiento de los números.</i>	7
<i>Figura 5. Salida por pantalla con la evolución de los datos en el TDA Vector.</i>	9

Bibliografía

- [1] B. W. Kernighan y D. M. Ritchie, El lenguaje de programación C, Pearson Educación, 1991.
- [2] H. M. Deitel y P. J. Deitel, Cómo programar en C/C+, Pearson Educación, 1995.
- [3] Herbert Schildt, Turbo C/C++ 3.1 Manual de referencia, Osborne/McGraw-Hill, 1994.